I. INTRODUCTION

In system security and data integrity preservation, monitoring critical directories and configuration files is an essential preventive measure. Detecting unauthorized file injections by malicious software or the manipulation of existing files must be achieved instantaneously.

The objective of this project is to develop a software prototype that operates with minimal background resource consumption, continuously scans a target local directory defined by the user, and immediately captures file-level events (such as file creation or modification) to generate security logs. The architecture of the system has been designed with flexibility to ensure that new modules, such as email notifications or database logging, can be seamlessly integrated in future iterations.

II. SOFTWARE ARCHITECTURE AND DESIGN PATTERN

The object-oriented structure of the application was built upon the **Observer Design Pattern** to ensure loose coupling between classes.

A. Class Structures and Roles

- **`IObserver` (Interface):** The abstract base class that forms the foundation of the notification mechanism. It contains a pure virtual `onNotify(const std::string& message)` function. This allows any new component intended to listen to the system to be integrated simply by inheriting from this class.
- **`SecurityLogger` (Observer Class):** Inherits publicly from the `IObserver` interface. It handles the triggered file events and outputs meaningful logs to the console screen with a `[SECURITY WARNING]` prefix.
- **`FileMonitor` (Subject / Main Monitor Class):** The central class responsible for tracking the directory. It stores the path of the target directory. Utilizing an internal `std::vector<IObserver*>` collection, it dynamically maintains the registered observers (`addObserver`).

B. Application of OOP Principles

Polymorphism is actively utilized within the system. The `FileMonitor` class does not need to know whether the notifying object is a `SecurityLogger` or another structural class; it executes the notification function strictly through an `IObserver` pointer. This approach enhances the abstraction level of the code and minimizes dependencies.

III. IMPLEMENTATION AND SYSTEM WORKFLOW

The structural definitions of the application are coded within the `Monitor.h` header file, while the core logic and loop execution reside in the `main.cpp` file. The runtime algorithm of the system operates as follows:

1. **Initial State Analysis:** When the `FileMonitor::startMonitoring()` function is invoked, the target directory is scanned using the `std::filesystem` library introduced in C++17. The absolute path and the last write time (`fs::last_write_time`) of each file are stored as key-value pairs in a `std::map<std::string, fs::file_time_type>` structure. This process establishes the baseline reference point for the application.
2. **Real-Time Scanning (Loop):** The application remains active through a continuous `while(true)` loop. To prevent excessive CPU utilization, the loop is configured to sleep for 1000 milliseconds (1 second) per iteration using `std::this_thread::sleep_for`.
3. **Change Detection Logic:** Every second, the directory is recursively traversed via `fs::recursive_directory_iterator`, and the instantaneous state is compared against the snapshot in the `map`:
    - If a scanned file path is not found within the `map`, it signifies that a new file has been created. The observers are notified, and the new path is inserted into the `map`.
    - If the file path exists but its last modification time differs from the record, it indicates that the file has been modified. The `onNotify` method is triggered to generate a log, and the timestamp is updated.

IV. CHALLENGES ENCOUNTERED AND TROUBLESHOOTING

During the coding and compilation phases of the project, two major practical issues originating from the Visual Studio IDE configuration and the C++ compiler design were encountered and resolved through the following steps:

A. Compiler Language Standard Mismatch and the MSB4086 Error

Because Visual Studio defaults to older C++ standards in its initial project settings, the `std::filesystem` namespace at the beginning of the code was not recognized by the compiler, resulting in an `E0135` (namespace has no member) error. When the language standard was switched to `ISO C++17 Standard (/std:c++17)` within the project properties, a numerical comparison conflict caused an `MSB4086` MSBuild error due to corrupted internal configuration files string evaluations.

Solution: To bypass this bottleneck, Visual Studio was closed, and the temporary `x64` build artifact directories within the project folder were deleted entirely. Upon restarting the IDE, the project properties were reverted to "Default" settings, and the C++17 standard was cleanly selected directly from the dropdown list instead of manual entry, followed by a full "Rebuild Solution". This resolved the compiler deadlock.

B. Forward Declaration and Dependency Ordering Issues

An `Error C2065: 'logger': undeclared identifier` error occurred due to the layout sequence of the classes in the `Monitor.h` file. Because the C++ compiler processes source code from top to bottom, the `FileMonitor` class was initially unable to recognize the `SecurityLogger` structure since it was defined further down.

Solution: The code architecture was reorganized according to the strict order of structural dependency. The abstract interface (`IObserver`) was positioned at the top, followed immediately by the concrete observer (`SecurityLogger`), and the primary `FileMonitor` class holding the pointers was placed at the bottom. This layout allowed the compiler to parse all classes in the correct context.

C. Directory Not Found Error and Runtime Adjustment

When the application was first executed, a runtime error stating "Error: Specified directory not found!" appeared on the console. The root cause was identified as the single backslash (`\`) character in the Windows file system path acting as an escape character in C++, breaking the string mapping to the desktop folder.

Solution: The path string within the `main` function was updated using double backslashes (`\\`) to properly escape the characters, matching the exact user profile directory layout: `C:\\Users\\ASUS1\\Desktop\\Ödev\\test`. This modification completely resolved the runtime path mismatch.

V. CONCLUSION

Within the scope of this study, a practical security tool was successfully implemented by combining modern C++ libraries with object-oriented design patterns. The use of the Observer pattern maximized architectural scalability. The application was thoroughly tested; when a new file was dropped into the target folder or an existing text document was modified, the corresponding logs were dispatched to the console within milliseconds. The project has provided valuable hands-on experience regarding compiler configurations and IDE synchronization issues frequently faced by software developers.

REFERENCES

[1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013. *(Core resource for C++ language fundamentals and standard library design)*

[2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. *(Reference for structural standards of the Observer design pattern used in the study)*

[3] ISO/IEC, "Information technology — Programming languages — C++", ISO/IEC Standard 14882:2017, 2017. *(Official C++17 specification for the `std::filesystem` library used throughout the project)*

[4] Microsoft Learn, "Directory monitoring and filesystem operations guide," Microsoft, 2024. [Online]. Available: https://learn.microsoft.com/en-us/cpp/standard-library/filesystem *(Official IDE documentation consulted to resolve compiler language standard conflicts and the MSB4086 build error)*

[5] S. Meyers, *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. O'Reilly Media, 2014. *(Reference for memory management and best practices regarding* `std::vector` *and pointers within the architecture)*